

Multi-Agent Reinforcement Learning & Agentic AI

Deep MARL

This lecture is based on

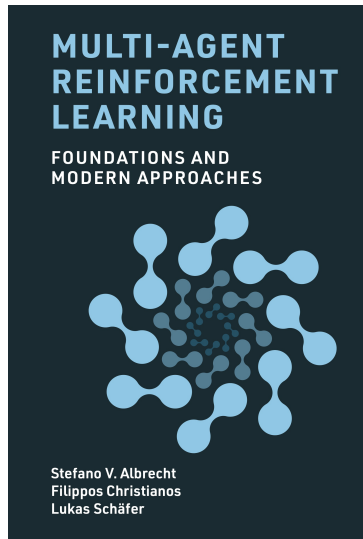
Multi-Agent Reinforcement Learning: Foundations and Modern Approaches

by Stefano V. Albrecht, Filippos Christianos and
Lukas Schäfer

MIT Press, 2024

Download book, slides, and code at:

www.marl-book.com



Lecture Outline

- Training and execution modes
- Multi-agent policy gradient algorithms
- Value decomposition in common-reward games
- Agent modelling with deep learning
- Population-based training

Training and Execution Modes

Training and Execution Modes

We often distinguish between training and execution modes in MARL:

- **Training:** what information is available to agents during learning?
- **Execution:** what information is available to agents for action selection?

Training and Execution Modes

We often distinguish between training and execution modes in MARL:

- **Training:** what information is available to agents during learning?
- **Execution:** what information is available to agents for action selection?

Reminder

We have already seen two single-agent reductions of the MARL problem:

Training and Execution Modes

We often distinguish between training and execution modes in MARL:

- **Training:** what information is available to agents during learning?
- **Execution:** what information is available to agents for action selection?

Reminder

We have already seen two single-agent reductions of the MARL problem:

- **Independent learning:** each agent learns its policy independently
→ decentralized training and decentralized execution

Training and Execution Modes

We often distinguish between training and execution modes in MARL:

- **Training:** what information is available to agents during learning?
- **Execution:** what information is available to agents for action selection?

Reminder

We have already seen two single-agent reductions of the MARL problem:

- **Independent learning:** each agent learns its policy independently
→ decentralized training and decentralized execution
- **Central learning:** learn single policy over the joint action space
→ centralized training and centralized execution

Training and Execution Modes

We often distinguish between training and execution modes in MARL:

- **Training:** what information is available to agents during learning?
- **Execution:** what information is available to agents for action selection?

Reminder

We have already seen two single-agent reductions of the MARL problem:

- **Independent learning:** each agent learns its policy independently
→ decentralized training and decentralized execution
- **Central learning:** learn single policy over the joint action space
→ centralized training and centralized execution

But we can also have **centralised training with decentralised execution (CTDE)**!

Training and Execution Modes

Decentralised Training

Centralised Training

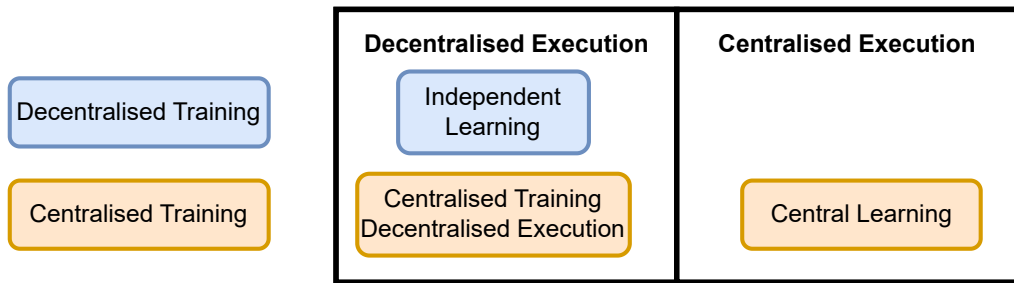
Decentralised Execution

Independent
Learning

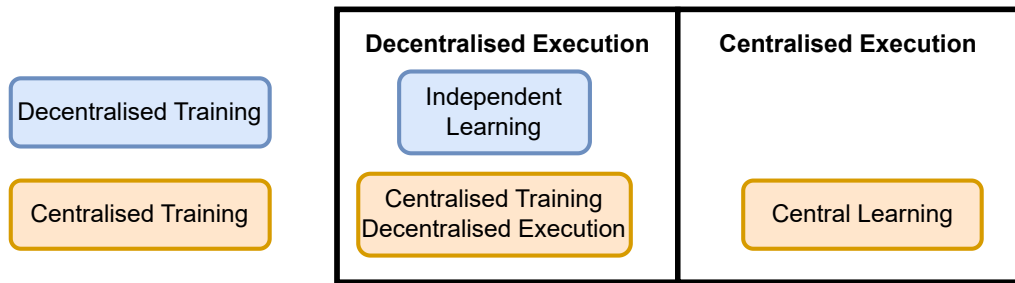
Centralised Execution

Central Learning

Training and Execution Modes



Training and Execution Modes



For multi-agent deep RL algorithms, we will focus on algorithms under decentralised execution.

Multi-Agent Policy Gradient Algorithms

The Policy-Gradient Theorem

Reminder

Follow this gradient to optimise the parameters ϕ of the policy π to maximise the expected return:

$$\begin{aligned}\nabla_{\phi} J(\phi) &\propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^{\pi}(s, a) \nabla_{\phi} \pi(a \mid s; \phi) \\ &= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi), a \sim \pi(\cdot \mid s; \phi)} [Q^{\pi}(s, a) \nabla_{\phi} \log \pi(a \mid s; \phi)]\end{aligned}$$

The Policy-Gradient Theorem

Reminder

Follow this gradient to optimise the parameters ϕ of the policy π to maximise the expected return:

$$\begin{aligned}\nabla_{\phi} J(\phi) &\propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^{\pi}(s, a) \nabla_{\phi} \pi(a \mid s; \phi) \\ &= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi), a \sim \pi(\cdot \mid s; \phi)} [Q^{\pi}(s, a) \nabla_{\phi} \log \pi(a \mid s; \phi)]\end{aligned}$$

Does this also hold for MARL? Yes, with minor modifications!

The Multi-Agent Policy-Gradient Theorem

Solution

In MARL, the expected returns of agent i under its policy π_i depends on the policies of all other agents $\pi_{-i} \rightarrow$ the multi-agent policy gradient theorem defines an expectation over the policies of **all** agents:

The Multi-Agent Policy-Gradient Theorem

Solution

In MARL, the expected returns of agent i under its policy π_i depends on the policies of all other agents $\pi_{-i} \rightarrow$ the multi-agent policy gradient theorem defines an expectation over the policies of **all** agents:

$$\nabla_{\phi_i} J(\phi_i) \propto \mathbb{E}_{\hat{h} \sim \Pr(\hat{h}|\pi), a_i \sim \pi_i, a_{-i} \sim \pi_{-i}} \left[Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle) \nabla_{\phi_i} \log \pi_i(a_i \mid h_i = \sigma_i(\hat{h}); \phi_i) \right]$$

The Multi-Agent Policy-Gradient Theorem

Solution

In MARL, the expected returns of agent i under its policy π_i depends on the policies of all other agents $\pi_{-i} \rightarrow$ the multi-agent policy gradient theorem defines an expectation over the policies of **all** agents:

$$\nabla_{\phi_i} J(\phi_i) \propto \mathbb{E}_{\hat{h} \sim \Pr(\hat{h}|\pi), a_i \sim \pi_i, a_{-i} \sim \pi_{-i}} \left[Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle) \nabla_{\phi_i} \log \pi_i(a_i \mid h_i = \sigma_i(\hat{h}); \phi_i) \right]$$

Derive policy update rules by finding estimators for expected return $Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle)$

The Multi-Agent Policy-Gradient Theorem

Solution

In MARL, the expected returns of agent i under its policy π_i depends on the policies of all other agents $\pi_{-i} \rightarrow$ the multi-agent policy gradient theorem defines an expectation over the policies of **all** agents:

$$\nabla_{\phi_i} J(\phi_i) \propto \mathbb{E}_{\hat{h} \sim \Pr(\hat{h}|\pi), a_i \sim \pi_i, a_{-i} \sim \pi_{-i}} \left[Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle) \nabla_{\phi_i} \log \pi_i(a_i \mid h_i = \sigma_i(\hat{h}); \phi_i) \right]$$

Derive policy update rules by finding estimators for expected return $Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle)$

We have already seen independent A2C that estimates $Adv(h_i, a_i) \approx Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle)$

The Multi-Agent Policy-Gradient Theorem

Solution

In MARL, the expected returns of agent i under its policy π_i depends on the policies of all other agents $\pi_{-i} \rightarrow$ the multi-agent policy gradient theorem defines an expectation over the policies of **all** agents:

$$\nabla_{\phi_i} J(\phi_i) \propto \mathbb{E}_{\hat{h} \sim \Pr(\hat{h}|\pi), a_i \sim \pi_i, a_{-i} \sim \pi_{-i}} \left[Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle) \nabla_{\phi_i} \log \pi_i(a_i \mid h_i = \sigma_i(\hat{h}); \phi_i) \right]$$

Derive policy update rules by finding estimators for expected return $Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle)$

We have already seen independent A2C that estimates $Adv(h_i, a_i) \approx Q_i^\pi(\hat{h}, \langle a_i, a_{-i} \rangle)$

But can we do better? Perhaps by leveraging more information?

Note

In actor-critic algorithms, only the policy/actor is used during execution and the critic is used only during training → the critic can be conditioned on centralised information z without compromising decentralised execution.

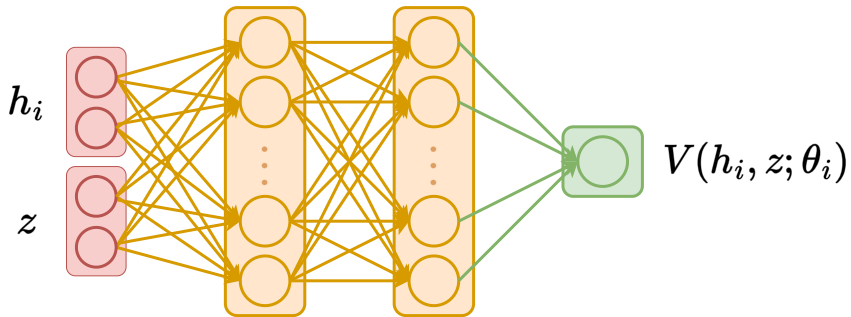
Note

In actor-critic algorithms, only the policy/actor is used during execution and the critic is used only during training → the critic can be conditioned on centralised information z without compromising decentralised execution.

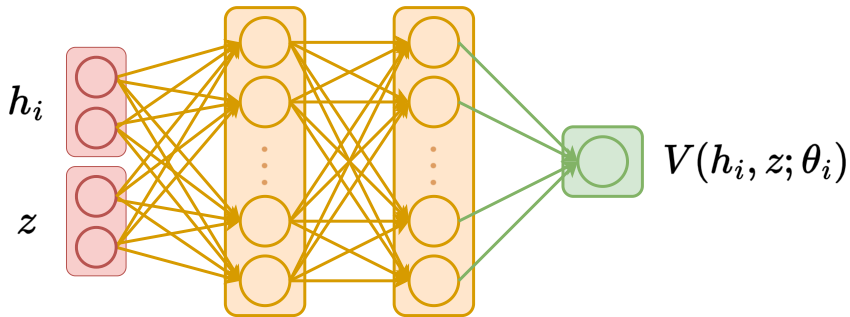
This might include:

- Global state s
- Joint action a
- Joint observation history h
- ...

Centralized Critics



Centralized Critics



Now we can integrate centralized critics into multi-agent policy gradient algorithms.

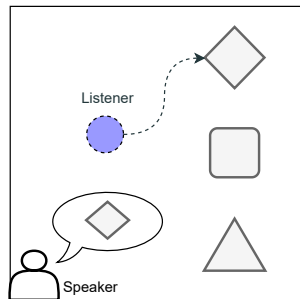
Centralized Advantage Actor-Critic

Algorithm Centralized A2C with synchronous environments

- 1: Initialize n actor networks with random parameters $\phi_1, \dots, \phi_n$
 - 2: Initialize n critic networks with random parameters $\theta_1, \dots, \theta_n$
 - 3: Initialize K parallel environments
 - 4: **for** time step $t = 0 \dots \mathbf{do}$
 - 5: Batch of observations for each agent and environment: $\begin{bmatrix} o_1^{t,1} \dots o_1^{t,K} \\ \vdots \\ o_n^{t,1} \dots o_n^{t,K} \end{bmatrix}$
 - 6: Batch of centralized information for each environment: $[z^{t,1} \dots z^{t,K}]$
 - 7: Sample actions $\begin{bmatrix} a_1^{t,1} \dots a_1^{t,K} \\ \vdots \\ a_n^{t,1} \dots a_n^{t,K} \end{bmatrix} \sim \pi(\cdot \mid h_1^t; \phi_1), \dots, \pi(\cdot \mid h_n^t; \phi_n)$
 - 8: Apply actions; collect rewards $\begin{bmatrix} r_1^{t,1} \dots r_1^{t,K} \\ \vdots \\ r_n^{t,1} \dots r_n^{t,K} \end{bmatrix}$, observations $\begin{bmatrix} o_1^{t+1,1} \dots o_1^{t+1,K} \\ \vdots \\ o_n^{t+1,1} \dots o_n^{t+1,K} \end{bmatrix}$, and centralized information $[z^{t+1,1} \dots z^{t+1,K}]$
 - 9: **for** agent $i = 1, \dots, n$ **do**
 - 10: **if** $s^{t+1,k}$ is terminal **then**
 - 11: $Adv(h_i^{t,k}, z^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} - V(h_i^{t,k}, z^{t,k}; \theta_i)$
 - 12: Critic target $y_i^{t,k} \leftarrow r_i^{t,k}$
 - 13: **else**
 - 14: $Adv(h_i^{t,k}, z^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}, z^{t+1,k}; \theta_i) - V(h_i^{t,k}, z^{t,k}; \theta_i)$
 - 15: Critic target $y_i^{t,k} \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}, z^{t+1,k}; \theta_i)$
 - 16: Actor loss $\mathcal{L}(\phi_i) \leftarrow \frac{1}{K} \sum_{k=1}^K Adv(h_i^{t,k}, z^{t,k}, a_i^{t,k}) \log \pi(a_i^{t,k} \mid h_i^{t,k}; \phi_i)$
 - 17: Critic loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{K} \sum_{k=1}^K (y_i^{t,k} - V(h_i^{t,k}, z^{t,k}; \theta_i))^2$
 - 18: Update parameters ϕ_i by minimizing the actor loss $\mathcal{L}(\phi_i)$
 - 19: Update parameters θ_i by minimizing the critic loss $\mathcal{L}(\theta_i)$
-

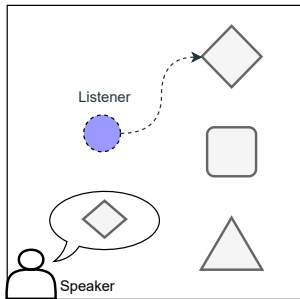
- Simple extension of independent A2C.
- Centralized information z is added to the critic input.

Centralized Critics in Action

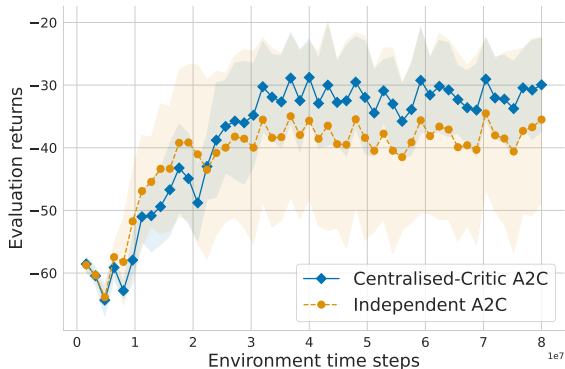


(a) Speaker-listener game

Centralized Critics in Action



(a) Speaker-listener game

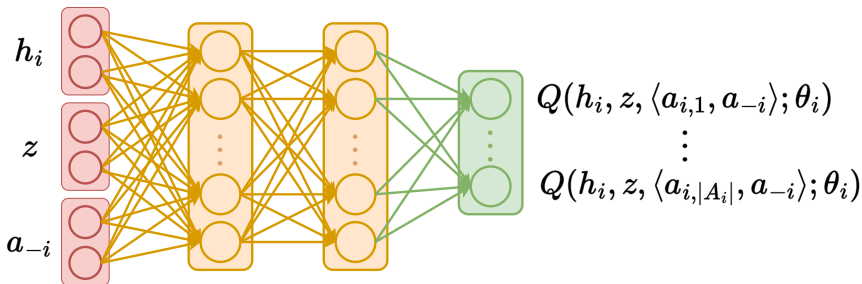


(b) Training curves

Agents with centralized critics converge to higher returns than agents with independent critics in the partially observable speaker-listener game.

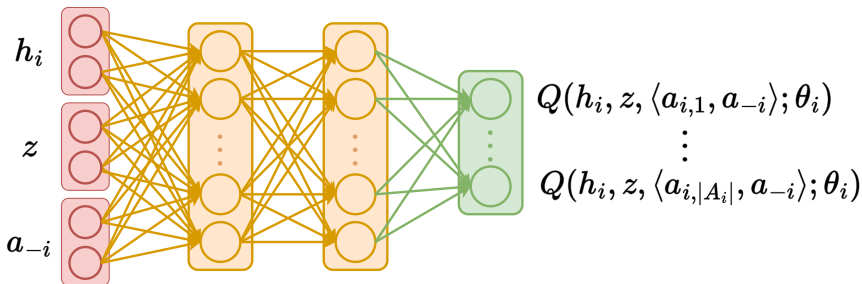
Centralized Action-Value Critics

Similarly, we can learn an action-value function that receives additional centralized information z .



Centralized Action-Value Critics

Similarly, we can learn an action-value function that receives additional centralized information z .



But what for? The centralized action-value critic can reason about the joint action space!

Counterfactual Multi-Agent Policy Gradient

For example, we can compute a counterfactual advantage for agent i

$$Adv_i(h_i, z, a) = Q(h_i, z, a; \theta) - \underbrace{\sum_{a'_i \in A_i} \pi(a'_i | h_i; \phi_i) Q(h_i, z, \langle a'_i, a_{-i} \rangle; \theta)}_{\text{counterfactual baseline}}$$

with the following components:

Counterfactual Multi-Agent Policy Gradient

For example, we can compute a counterfactual advantage for agent i

$$Adv_i(h_i, z, a) = Q(h_i, z, a; \theta) - \underbrace{\sum_{a'_i \in A_i} \pi(a'_i | h_i; \phi_i) Q(h_i, z, \langle a'_i, a_{-i} \rangle; \theta)}_{\text{counterfactual baseline}}$$

with the following components:

- $Q(h_i, z, a; \theta)$: expected returns when applying joint action $a \rightarrow$ agent i applies action a_i and all other agents apply actions a_{-i}

Counterfactual Multi-Agent Policy Gradient

For example, we can compute a counterfactual advantage for agent i

$$Adv_i(h_i, z, a) = Q(h_i, z, a; \theta) - \underbrace{\sum_{a'_i \in A_i} \pi(a'_i | h_i; \phi_i) Q(h_i, z, \langle a'_i, a_{-i} \rangle; \theta)}_{\text{counterfactual baseline}}$$

with the following components:

- $Q(h_i, z, a; \theta)$: expected returns when applying joint action $a \rightarrow$ agent i applies action a_i and all other agents apply actions a_{-i}
- Counterfactual baseline: expected returns when agent i samples its action a'_i from its policy $\pi(\cdot; \phi_i)$ and all other agents apply their actual actions a_{-i}

Counterfactual Multi-Agent Policy Gradient

For example, we can compute a counterfactual advantage for agent i

$$Adv_i(h_i, z, a) = Q(h_i, z, a; \theta) - \underbrace{\sum_{a'_i \in A_i} \pi(a'_i | h_i; \phi_i) Q(h_i, z, \langle a'_i, a_{-i} \rangle; \theta)}_{\text{counterfactual baseline}}$$

with the following components:

- $Q(h_i, z, a; \theta)$: expected returns when applying joint action $a \rightarrow$ agent i applies action a_i and all other agents apply actions a_{-i}
- Counterfactual baseline: expected returns when agent i samples its action a'_i from its policy $\pi(\cdot; \phi_i)$ and all other agents apply their actual actions a_{-i}

Identify contribution of agent i 's action a_i to received rewards $\Rightarrow$ help to address the **credit assignment problem** in common-reward games

Centralized Critic Algorithms

Commonly seen centralized critic algorithms:

- MAA2C: A2C agent with a centralized critic
- COMA: MAA2C with Counterfactual Multi-Agent Policy Gradients
- MADDPG: DDPG agent with a centralized Q
- MATRPO: TRPO agent with a centralized critic
- MAPPO: PPO agent with a centralized critic
- HATRPO: Sequentially updating critic of MATRPO agents
- HAPPO: Sequentially updating critic of MAPPO agents

Link to tutorial: <https://pettingzoo.farama.org/tutorials/agilerl/MADDPG/>

Value Decomposition in Common-Reward Games

Centralized Value Functions in Value-Based MARL

We addressed MARL challenges in policy gradient algorithms by using centralized critics.
Can we also use centralized value functions in value-based MARL algorithms?

Centralized Value Functions in Value-Based MARL

We addressed MARL challenges in policy gradient algorithms by using centralized critics. Can we also use centralized value functions in value-based MARL algorithms?

Problem

In value-based MARL algorithms (e.g. IDQN) agents learn value functions and derive their policy from them. However, learning and deriving a policy from centralized value functions would prevent decentralized execution.

Centralized Value Functions in Value-Based MARL

We addressed MARL challenges in policy gradient algorithms by using centralized critics. Can we also use centralized value functions in value-based MARL algorithms?

Problem

In value-based MARL algorithms (e.g. IDQN) agents learn value functions and derive their policy from them. However, learning and deriving a policy from centralized value functions would prevent decentralized execution.

How can we overcome this limitation and leverage the benefits of centralized value functions in value-based MARL algorithms?

Value Decomposition

We will focus on **value decomposition** methods for common-reward games. These methods aim to decompose a centralized action-value function of all agents

$$Q(h^t, z^t, a^t; \theta) = \mathbb{E} \left[\sum_{\tau=t}^{\infty} \gamma^{\tau-t} r^{\tau} \mid h^t, z^t, a^t \right]$$

into individual utility functions of each agent: $Q(h_i, a_i; \theta_i)$ for $i \in I$

Value Decomposition

We will focus on **value decomposition** methods for common-reward games. These methods aim to decompose a centralized action-value function of all agents

$$Q(h^t, z^t, a^t; \theta) = \mathbb{E} \left[\sum_{\tau=t}^{\infty} \gamma^{\tau-t} r^{\tau} \mid h^t, z^t, a^t \right]$$

into individual utility functions of each agent: $Q(h_i, a_i; \theta_i)$ for $i \in I$

This decomposition has several benefits:

- Agents benefit from centralized information during training
- Simplify learning by decomposing the centralized value function
- Agents learn their individual utility functions to represent their contribution to the centralized value function, helping to address the **credit assignment problem**

Individual-Global-Max Property

How to ensure that decentralized action selection with respect to the agents' individual utility functions leads to effective joint actions with respect to the centralized action-value function?

Individual-Global-Max Property

How to ensure that decentralized action selection with respect to the agents' individual utility functions leads to effective joint actions with respect to the centralized action-value function?

Solution

Let $\hat{h}$ be a full history with joint-observation histories $h = \sigma(\hat{h})$, individual observation histories, $h_i = \sigma_i(\hat{h})$, and centralized information z . The **individual-global-max (IGM) property** is satisfied if and only if:

$$\forall a = (a_1, \dots, a_n) \in A : a \in A^*(h, z; \theta) \iff \forall i \in I : a_i \in A_i^*(h_i; \theta_i)$$

$$\text{with } A^*(h, z; \theta) = \arg \max_{a \in A} Q(h, z, a; \theta) \quad \text{and} \quad A_i^*(h_i; \theta_i) = \arg \max_{a_i \in A_i} Q(h_i, a_i; \theta_i).$$

The Importance of the Individual-Global-Max Property

Upholding the IGM property has two important implications:

The Importance of the Individual-Global-Max Property

Upholding the IGM property has two important implications:

1. If all agents decentrally select actions that maximise their individual utility functions, the resulting joint action will maximise the centralized action-value function → effective decentralised execution

The Importance of the Individual-Global-Max Property

Upholding the IGM property has two important implications:

1. If all agents decentrally select actions that maximise their individual utility functions, the resulting joint action will maximise the centralized action-value function → **effective decentralised execution**
2. The greedy joint action with respect to the centralized action-value function can be efficiently obtained by selecting the greedy action for each agent with respect to their individual utility functions → **efficient centralized training**

The Importance of the Individual-Global-Max Property

Upholding the IGM property has two important implications:

1. If all agents decentrally select actions that maximise their individual utility functions, the resulting joint action will maximise the centralized action-value function → **effective decentralised execution**
2. The greedy joint action with respect to the centralized action-value function can be efficiently obtained by selecting the greedy action for each agent with respect to their individual utility functions → **efficient centralized training**

Note

It is not guaranteed that for a given environment, there exists a decomposition of the centralized action-value function that satisfies the IGM property.

Linear Value Decomposition

Value decomposition networks (VDN) uses a simple linear decomposition of the centralized action-value function:

$$Q(h^t, z^t, a^t; \theta) = \sum_{i \in I} Q(h_i^t, a_i^t; \theta_i)$$

Linear Value Decomposition

Value decomposition networks (VDN) uses a simple linear decomposition of the centralized action-value function:

$$Q(h^t, z^t, a^t; \theta) = \sum_{i \in I} Q(h_i^t, a_i^t; \theta_i)$$

This decomposition satisfies the IGM property and we can jointly optimise the parameters of all networks by minimising the following loss on sampled batches of experiences $\mathcal{B}$:

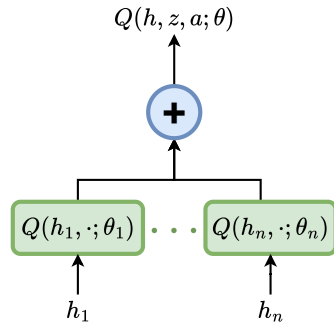
$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{(h^t, a^t, r^t, h^{t+1}) \in \mathcal{B}} \left(r^t + \gamma \sum_{i \in I} \max_{a_i \in A_i} Q(h_i^{t+1}, a_i; \bar{\theta}_i) - \sum_{i \in I} Q(h_i^t, a_i^t; \theta_i) \right)^2$$

with $\bar{\theta}_i$ denoting the parameters of agent i 's target network.

Value Decomposition Networks

Algorithm Value decomposition networks (VDN)

- 1: Initialize n utility networks with random parameters $\theta_1, \dots, \theta_n$
 - 2: Initialize n target networks with parameters $\bar{\theta}_1 = \theta_1, \dots, \bar{\theta}_n = \theta_n$
 - 3: Initialize a shared replay buffer D
 - 4: **for** time step $t = 0, 1, 2, \dots$ **do**
 - 5: Collect current observations $o_1^t, \dots, o_n^t$
 - 6: **for** agent $i = 1, \dots, n$ **do**
 - 7: With probability ϵ : choose random action a_i^t
 - 8: Otherwise: choose $a_i^t \in \arg \max_{a_i} Q(h_i^t, a_i; \theta_i)$
 - 9: Apply actions; collect shared reward r^t and next observations $o_1^{t+1}, \dots, o_n^{t+1}$
 - 10: Store transition (h^t, a^t, r^t, h^{t+1}) in shared replay buffer D
 - 11: Sample mini-batch of B transitions (h^k, a^k, r^k, h^{k+1}) from D
 - 12: **if** s^{k+1} is terminal **then**
 - 13: Targets $y^k \leftarrow r^k$
 - 14: **else**
 - 15: Targets $y^k \leftarrow r^k + \gamma \sum_{i \in I} \max_{a_i' \in A_i} Q(h_i^{k+1}, a_i'; \bar{\theta}_i)$
 - 16: Loss $\mathcal{L}(\theta) \leftarrow \frac{1}{B} \sum_{k=1}^B \left(y^k - \sum_{i \in I} Q(h_i^k, a_i^k; \theta_i) \right)^2$
 - 17: Update parameters θ by minimizing the loss $\mathcal{L}(\theta)$
 - 18: In a set interval, update target network parameters $\bar{\theta}_i$ for each agent i
-



Monotonic Value Decomposition

A more general decomposition (that also ensures the IGM property) can be formulated by assuming that the centralized action-value function is a (strictly) monotonically increasing function with respect to any individual utility function:

$$\forall i \in I, \forall a \in A: \frac{\partial Q(h, z, a; \theta)}{\partial Q(h_i, a_i; \theta_i)} > 0$$

Monotonic Value Decomposition

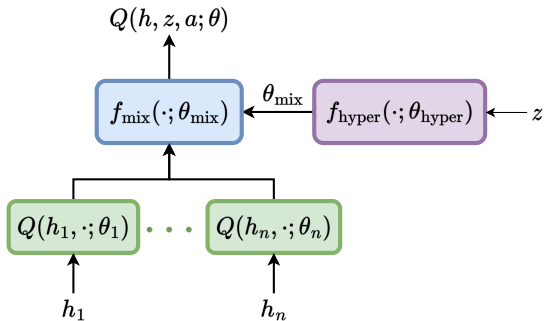
A more general decomposition (that also ensures the IGM property) can be formulated by assuming that the centralized action-value function is a (strictly) monotonically increasing function with respect to any individual utility function:

$$\forall i \in I, \forall a \in A : \frac{\partial Q(h, z, a; \theta)}{\partial Q(h_i, a_i; \theta_i)} > 0$$

The **QMIX** algorithm implements this assumption using a mixing function f_{mix} that aggregates individual utilities to approximate the centralized action-value function:

$$Q(h, z, a, \theta) = f_{\text{mix}}(Q(h_1, a_1; \theta_1), \dots, Q(h_n, a_n; \theta_n); \theta_{\text{mix}})$$

QMIX Architecture



The centralized action-value function is monotonic with respect to individual utilities if all weights of f_{mix} are positive

$\Rightarrow$ Ensure positive weights by obtaining the parameters of the mixing function from a hypernetwork f_{hyper} conditioned on centralized information z

QMIX Optimisation

Parameters of all utility functions and hypernetwork are jointly optimised by minimising the following loss on batches of experiences $\mathcal{B}$ sampled from a replay buffer:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{(h^t, z^t, a^t, r^t, h^{t+1}, z^{t+1}) \in \mathcal{B}} \left(r^t + \gamma \max_{a \in A} Q(h^{t+1}, z^{t+1}, a; \bar{\theta}) - Q(h^t, z^t, a^t; \theta) \right)^2$$

with the following decomposed value estimates:

$$Q(h^t, z^t, a^t, \theta) = f_{\text{mix}} \left(Q(h_1^t, a_1^t; \theta_1), \dots, Q(h_n^t, a_n^t; \theta_n); \theta_{\text{mix}} \right)$$
$$\max_{a \in A} Q(h^{t+1}, z^{t+1}, a; \bar{\theta}) = f_{\text{mix}} \left(\max_{a_1 \in A_1} Q(h_1^{t+1}, a_1; \bar{\theta}_1), \dots, \max_{a_n \in A_n} Q(h_n^{t+1}, a_n; \bar{\theta}_n); \bar{\theta}_{\text{mix}} \right)$$

Value Decomposition in Matrix Games

To better understand how value decomposition works in practise, we will look at several exemple tasks and the learned decompositions of both VDN and QMIX.

Value Decomposition in Matrix Games

To better understand how value decomposition works in practise, we will look at several exemple tasks and the learned decompositions of both VDN and QMIX.

	A	B
A	1	5
B	5	9

Figure: Linear game

	A	B
A	0	0
B	0	10

Figure: Monotonic game

	A	B	C
A	11	-30	0
B	-30	7	0
C	0	6	5

Figure: Climbing game

Value Decomposition in Matrix Games

To better understand how value decomposition works in practise, we will look at several exemple tasks and the learned decompositions of both VDN and QMIX.

	A	B
A	1	5
B	5	9

Figure: Linear game

	A	B
A	0	0
B	0	10

Figure: Monotonic game

	A	B	C
A	11	-30	0
B	-30	7	0
C	0	6	5

Figure: Climbing game

	Linear game	Monotonic game	Climbing game
Linearly decomposable	✓	✗	✗
Monotonically decomposable	✓	✓	✗

Value Decomposition in Linearly Decomposable Matrix Game

	A	B
A	1	5
B	5	9

(a) True rewards

	0.12	4.12
0.88	1.00	5.00
4.88	5.00	9.00

(b) VDN decomposition

	-0.21	0.68
0.19	1.00	5.00
0.96	5.00	9.00

(c) QMIX decomposition

Value Decomposition in Linearly Decomposable Matrix Game

	A	B
A	1	5
B	5	9

(a) True rewards

	0.12	4.12
0.88	1.00	5.00
4.88	5.00	9.00

(b) VDN decomposition

	-0.21	0.68
0.19	1.00	5.00
0.96	5.00	9.00

(c) QMIX decomposition

We can make several observations from the learned decompositions:

Value Decomposition in Linearly Decomposable Matrix Game

	A	B
A	1	5
B	5	9

(a) True rewards

	0.12	4.12
0.88	1.00	5.00
4.88	5.00	9.00

(b) VDN decomposition

	-0.21	0.68
0.19	1.00	5.00
0.96	5.00	9.00

(c) QMIX decomposition

We can make several observations from the learned decompositions:

- VDN and QMIX are able to learn the true centralized action-value function

Value Decomposition in Linearly Decomposable Matrix Game

	A	B
A	1	5
B	5	9

(a) True rewards

	0.12	4.12
0.88	1.00	5.00
4.88	5.00	9.00

(b) VDN decomposition

	-0.21	0.68
0.19	1.00	5.00
0.96	5.00	9.00

(c) QMIX decomposition

We can make several observations from the learned decompositions:

- VDN and QMIX are able to learn the true centralized action-value function
- The learned decompositions are not unique and can vary between different runs

Value Decomposition in Linearly Decomposable Matrix Game

	A	B
A	1	5
B	5	9

(a) True rewards

	0.12	4.12
0.88	1.00	5.00
4.88	5.00	9.00

(b) VDN decomposition

	-0.21	0.68
0.19	1.00	5.00
0.96	5.00	9.00

(c) QMIX decomposition

We can make several observations from the learned decompositions:

- VDN and QMIX are able to learn the true centralized action-value function
- The learned decompositions are not unique and can vary between different runs
- Individual utility values, in particular for QMIX, can be difficult to interpret (besides larger values indicating higher return estimates)

Value Decomposition in Monotonically Decomposable Matrix Game

	A	B
A	0	0
B	0	10

(a) True rewards

	-1.45	3.45
-0.94	-2.43	2.51
4.08	2.60	7.53

(b) VDN decomposition

	-4.91	0.82
-4.66	0.00	0.00
1.81	0.00	10.00

(c) QMIX decomposition

Value Decomposition in Monotonically Decomposable Matrix Game

	A	B
A	0	0
B	0	10

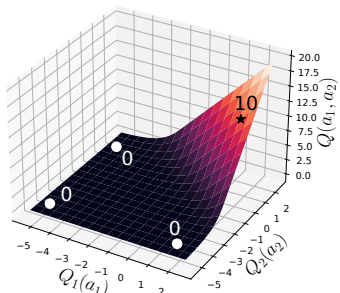
(a) True rewards

	-1.45	3.45
-0.94	-2.43	2.51
4.08	2.60	7.53

(b) VDN decomposition

	-4.91	0.82
-4.66	0.00	0.00
1.81	0.00	10.00

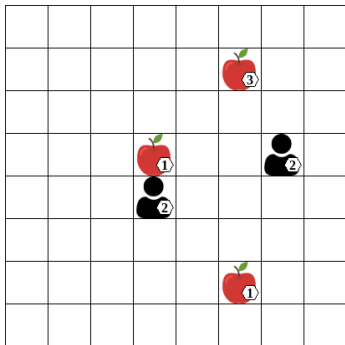
(c) QMIX decomposition



Only QMIX is able to represent the non-linear but monotonic relationship between the individual utility functions and the centralized action-value function.

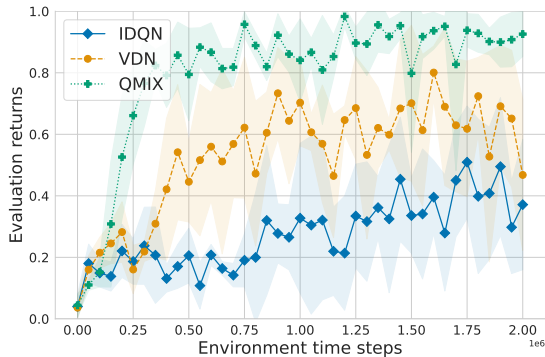
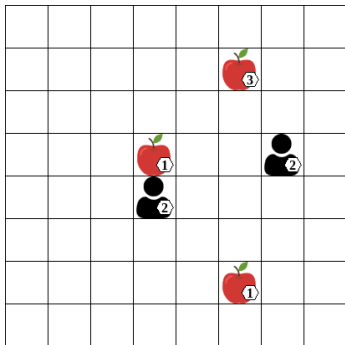
Value Decomposition in LBF

Now compare IDQN, VDN and QMIX in a common-reward level-based foraging task where two agents need to collect three items:



Value Decomposition in LBF

Now compare IDQN, VDN and QMIX in a common-reward level-based foraging task where two agents need to collect three items:



Value Decomposition Algorithms

Commonly seen value decomposition algorithms:

- VDN: mixing Q with value decomposition network
- QMIX: mixing Q with monotonic factorization
- FACMAC: mixing a bunch of DDPG agents
- VDA2C: mixing a bunch of A2C agents' critics
- VDPPPO: mixing a bunch of PPO agents' critics

Agent Modeling with Deep Learning

Agents Modeling – Motivation

In MARL, agents need to consider the policies of other agents to coordinate their actions.

Agents Modeling – Motivation

In MARL, agents need to consider the policies of other agents to coordinate their actions.

Approaches presented so far account for the action selection of other agents through:

- Distribution of training data is dependent on the policies of all agents
- Training centralized critics conditioned on the actions of other agents

Agents Modeling – Motivation

In MARL, agents need to consider the policies of other agents to coordinate their actions.

Approaches presented so far account for the action selection of other agents through:

- Distribution of training data is dependent on the policies of all agents
- Training centralized critics conditioned on the actions of other agents

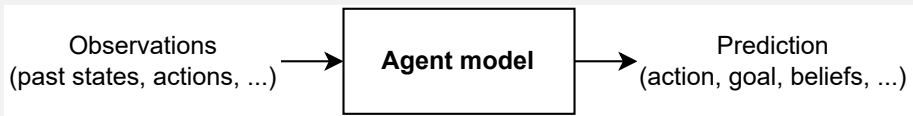
Can we provide agents with more **explicit** information about the policies of other agents so they can learn to coordinate better, e.g. by learning best-response policies?

Recap: Agent Modeling

Reminder

In the previous lectures, we have seen approaches that model other agents' policies:

- Learn models of other agents to predict their actions
- Compute optimal action (best-response) against agent models



S. Albrecht, P. Stone. **Autonomous Agents Modelling Other Agents: A Comprehensive Survey and Open Problems.** *Artificial Intelligence*, 2018

Recap: Joint-Action Learning with Agent Models

Reminder

We have already seen **joint-action learning with agent models** (JAL-AM)

Recap: Joint-Action Learning with Agent Models

Reminder

We have already seen **joint-action learning with agent models** (JAL-AM)

- Agents learn models of other agents conditioned on states s : $\hat{\pi}_{-i}(a_{-i} \mid s)$

Recap: Joint-Action Learning with Agent Models

Reminder

We have already seen **joint-action learning with agent models** (JAL-AM)

- Agents learn models of other agents conditioned on states s : $\hat{\pi}_{-i}(a_{-i} \mid s)$
- Agents learn value functions conditioned on the joint action: $Q_i(s, a)$

Recap: Joint-Action Learning with Agent Models

Reminder

We have already seen **joint-action learning with agent models** (JAL-AM)

- Agents learn models of other agents conditioned on states s : $\hat{\pi}_{-i}(a_{-i} \mid s)$
- Agents learn value functions conditioned on the joint action: $Q_i(s, a)$
- Using the value function and agent models, agent i can compute its expected action values under the current models of other agents:

$$AV_i(s, a_i) = \sum_{a_{-i} \in A_{-i}} Q_i(s, \langle a_i, a_{-i} \rangle) \prod_{j \in I \setminus \{i\}} \hat{\pi}_j(a_j \mid s)$$

Recap: Joint-Action Learning with Agent Models

Reminder

We have already seen **joint-action learning with agent models** (JAL-AM)

- Agents learn models of other agents conditioned on states s : $\hat{\pi}_{-i}(a_{-i} \mid s)$
- Agents learn value functions conditioned on the joint action: $Q_i(s, a)$
- Using the value function and agent models, agent i can compute its expected action values under the current models of other agents:

$$AV_i(s, a_i) = \sum_{a_{-i} \in A_{-i}} Q_i(s, \langle a_i, a_{-i} \rangle) \prod_{j \in I \setminus \{i\}} \hat{\pi}_j(a_j \mid s)$$

- Use AV_i to select optimal actions and as learning update targets

Joint-Action Learning with Deep Agent Models

How to do this with deep learning for partially observable environments (POSG)?

Joint-Action Learning with Deep Agent Models

How to do this with deep learning for partially observable environments (POSG)?

- Represent agent i 's model of agent j as a neural network: $\hat{\pi}_j^i(a_j \mid h_i; \phi_j^i)$
- Given the observation history of agent i , its model $\hat{\pi}_j^i(a_j \mid h_i; \phi_j^i)$ for agent j outputs a probability distribution over actions

Joint-Action Learning with Deep Agent Models

How to do this with deep learning for partially observable environments (POSG)?

- Represent agent i 's model of agent j as a neural network: $\hat{\pi}_j^i(a_j | h_i; \phi_j^i)$
- Given the observation history of agent i , its model $\hat{\pi}_j^i(a_j | h_i; \phi_j^i)$ for agent j outputs a probability distribution over actions
- Agent i can train its model for agent j by minimizing the cross-entropy loss between the predicted policy $\hat{\pi}_j^i$ and the observed actions of agent j :

$$\mathcal{L}(\phi_j^i) = \mathbb{E}_{a_j^t \sim \pi_j(h_j^t)} [-\log \hat{\pi}_j^i(a_j^t | h_i^t; \phi_j^i)]$$

Joint-Action Learning with Deep Agent Models

How to do this with deep learning for partially observable environments (POSG)?

- Represent agent i 's model of agent j as a neural network: $\hat{\pi}_j^i(a_j | h_i; \phi_j^i)$
- Given the observation history of agent i , its model $\hat{\pi}_j^i(a_j | h_i; \phi_j^i)$ for agent j outputs a probability distribution over actions
- Agent i can train its model for agent j by minimizing the cross-entropy loss between the predicted policy $\hat{\pi}_j^i$ and the observed actions of agent j :

$$\mathcal{L}(\phi_j^i) = \mathbb{E}_{a_j^t \sim \pi_j(h_j^t)} [-\log \hat{\pi}_j^i(a_j^t | h_i^t; \phi_j^i)]$$

- Then, agent i can compute expected action values:

$$AV(h_i, a_i; \theta_i) = \sum_{a_{-i} \in A_{-i}} Q(h_i, \langle a_i, a_{-i} \rangle; \theta_i) \prod_{j \neq i} \hat{\pi}_j^i(a_j | h_i; \phi_j^i)$$

Learning Compact Representations of Agent Policies

- JAL-AM combines agent models and centralized value functions to compute best-response policies.
- Can we integrate agent models into multi-agent policy gradient algorithms, e.g. by conditioning policies on agent models?

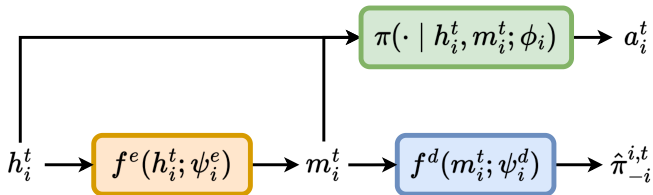
Learning Compact Representations of Agent Policies

- JAL-AM combines agent models and centralized value functions to compute best-response policies.
- Can we integrate agent models into multi-agent policy gradient algorithms, e.g. by conditioning policies on agent models?

Problem

To condition policies (and value functions) of agents on the policies of other agents, we need **compact** representations of the policies of other agents. How can we learn such representations?

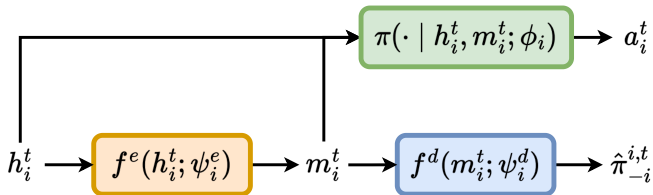
Learning Compact Representations of Agent Policies



Agent i trains encoder-decoder architecture with

- **Encoder f^e** with parameters ψ_i^e : given observation history h_i^t of agent i , output compact representation m_i^t of the policies of other agents
- **Decoder f^d** with parameters ψ_i^d : given compact representation m_i^t , predict the policies $\hat{\pi}_{-i}^{i,t}$ of other agents

Learning Compact Representations of Agent Policies



Agent i trains encoder-decoder architecture with

- **Encoder f^e** with parameters ψ_i^e : given observation history h_i^t of agent i , output compact representation m_i^t of the policies of other agents
- **Decoder f^d** with parameters ψ_i^d : given compact representation m_i^t , predict the policies $\hat{\pi}_{-i}^{i,t}$ of other agents

Then, agent i can condition its policy on the compact representations m_i^t .

Learning Compact Representations of Agent Policies

The encoder and decoder are jointly trained to minimize the cross-entropy loss for the predicted action probabilities and true actions of all other agents:

$$\mathcal{L}(\psi_i^e, \psi_i^d) = \sum_{j \neq i} -\log \hat{\pi}_j^{i,t}(a_j^t) \quad \text{with} \quad \hat{\pi}_j^{i,t} = f^d \left(f^e(h_i^t; \psi_i^e); \psi_i^d \right)_j$$

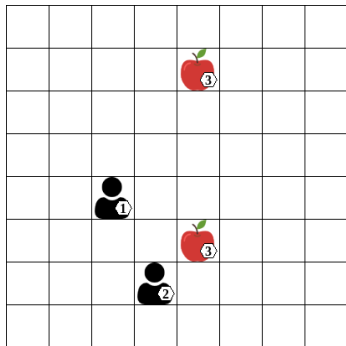
Learning Compact Representations of Agent Policies

The encoder and decoder are jointly trained to minimize the cross-entropy loss for the predicted action probabilities and true actions of all other agents:

$$\mathcal{L}(\psi_i^e, \psi_i^d) = \sum_{j \neq i} -\log \hat{\pi}_j^{i,t}(a_j^t) \quad \text{with} \quad \hat{\pi}_j^{i,t} = f^d \left(f^e(h_i^t; \psi_i^e); \psi_i^d \right)_j$$

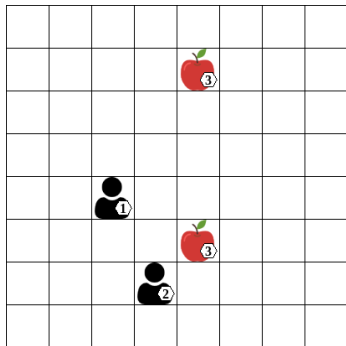
Encoder-decoder agent models can be integrated into MARL algorithm by conditioning value functions and policies on the obtained policy representations.

Compact Agent Policy Representations in LBF

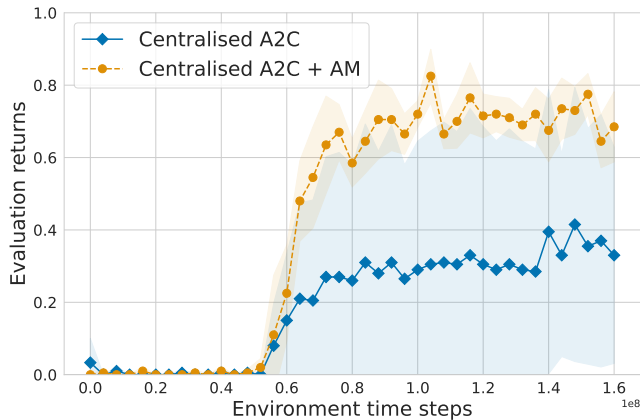


(a) Environment

Compact Agent Policy Representations in LBF



(a) Environment



(b) Learning curve

Summary

We covered:

- Centralised learning and execution
- Decentralised learning and execution (independent learning)
- Centralised learning and decentralised execution
 - Multi-agent policy gradient
 - Value decomposition
 - Joint-Action Learning with Deep Agent Models

Resources:

- <https://pettingzoo.farama.org/>
- <https://marllib.readthedocs.io/en/latest/index.html>